# ECE 264 - Object-Oriented Software Development

1. Introduction to OOP

# What is Object-Oriented Programming?

- Object-Oriented Programming (OOP) combines related variables (properties) and functions (methods) into objects, improving code organization and reusability.
- Key OOP concepts
  - Encapsulation - Groups related data and functions
  - Abstraction - Hides complexity
  - Inheritance - Eliminates redundant code
  - Polymorphism - Allows objects to behave differently based on their type.

# Why?

- OOP addresses the limitations of procedural programming
  - Functions and data are separate, leading to complex and hard-to-maintain code.
  - Procedural Programming: Emphasizes sequences of instructions (e.g., C).

Key Procedural Programming

- Top-down approach
- Procedures/Functions
- Separation of data and functions
- Code is AI generated

```c
#include <stdio.h>

// Function declaration (procedure definition)
// This function takes an integer 'n' and returns its factorial.
int calculate_factorial(int n) {
    // Local variable to store the result
    int result = 1;

    // Loop to multiply numbers from 1 to n
    for (int i = 1; i <= n; i++) {
        result *= i; // Same as result = result * i;
    }

    // Return the final result
    return result;
}

// The main function where program execution begins (the main procedure)
int main() {
    // Local variables in the main procedure
    int number_to_calculate;
    int final_result;

    // 1. Get input from the user (an instruction)
    printf("Enter a positive integer: ");
    scanf("%d", &number_to_calculate);

    // 2. Call the 'calculate_factorial' function, passing data to it
    final_result = calculate_factorial(number_to_calculate);

    // 3. Display the result (another instruction)
    printf("The factorial of %d is %d\n", number_to_calculate, final_resu

    // End of the program
    return 0;
}
```

## OOP - Key Differences

- Data and functions combined
- Encapsulation
- Object interaction
- Code is AI generated

python

```python
class FactorialCalculator:
    """A class to calculate the factorial of a given number using OOP

    def __init__(self, number):
        """Initializes the object with a number attribute."""
        # Encapsulates the data (the number) within the object
        self.number = number
        self._result = None # Use a leading underscore to suggest 'pro

    def calculate(self):
        """Calculates the factorial and stores it internally."""
        if self.number < 0:
            return "Error: Factorial does not exist for negative numbe
        elif self.number == 0:
            self._result = 1
        else:
            fact = 1
            # The method uses the encapsulated data (self.number)
            for i in range(1, self.number + 1):
                fact *= i
            self._result = fact
        return self._result

    def get_result_message(self):
        """Provides a formatted message for the result (Abstraction)."
        if self._result is not None and isinstance(self._result, int):
            return f"The factorial of {self.number} is {self._result}"
        else:
            return self._result

# --- Main program flow using the OOP class ---

# 1. Instantiate an object (create an instance of the class)
user_input_str = input("Enter a positive integer: ")
user_number = int(user_input_str)

# 2. Interact with the object via its methods
calculator_object = FactorialCalculator(user_number)
result = calculator_object.calculate()

# 3. Use another method to present the result
print(calculator_object.get_result_message())
```

# Key Building Blocks

- Classes
  - A class is a user-defined blueprint or template for creating objects.
  - It defines the standard attributes and methods that all instances of that clas will possess
- Objects
  - An object is a specific instance of a class
  - E.g., if car is a class, then your personal car, with its specific color, model, and speed, is an object (or instance) of that class.

# Four Pillars

- **Encapsulation** - A core principle of OOP
  - Bundles data and the functions that operate on it within an object. This approach reduces the number of parameters needed in functions, as the data is already part of the object.
- **Abstraction** simplifies the use of objects by hiding internal complexities and exposing only essential information.
- **Inheritance** allows new objects to inherit properties and methods from existing ones, promoting code reuse and reducing redundancy.
- **Polymorphism** enables objects of different classes to respond to the same method call in their own specific ways, eliminating the need for lengthy conditional statements.

# Popular OOP Language

- Python
- Java
- C++
- C#
- Ruby

# What is Python?

- Python is a high-level, **general-purpose programming language** known for its clear, English-like syntax and emphasis on code **readability**
- Key Characteristics
  - Interpreted
  - Multi-Paradigm
  - Dynamically Typed
  - Extensive Standard Library
  - Open-Source and Portable

# Course Outline

- Python Fundamentals
  - Variables and Data Types
  - Control Flow
  - Data Structures
  - Functions
- Introduction to Classes and Objects
- Implementation of the four pillars in Python